UNIVERSIDAD DE SAN BUENAVENTURA

TECNOLOGÍA EN DESARROLLO DE SOFTWARE | OPTATIVA 2

DOCUMENTO TÉCNICO

Elmercado10

Arquitectura de Software

Capas, componentes y decisiones de diseño

AUTORES Juan Esteban Olaya Muñoz
Nicolas Quiroz Gomez
Juan Sebastian Garzon Suarez

DOCENTE Jairo Armando Salcedo

ASIGNATURA Optativa 2

PROGRAMA Tecnología en Desarrollo de Software

INSTITUCIÓN Universidad de San Buenaventura

FECHA 3 de noviembre de 2026

Tabla de contenido

1. Visión general 3
2. Capas 3
3. Decisiones de diseño 4
4. Flujos principales 5
5. Despliegue 6
6. Ciclo de vida de los ViewModels 6

1. Visión general

Elmercado10 es una aplicación Android nativa sin servidor propio. La lógica de presentación y de negocio se ejecuta en el dispositivo y los datos residen en Firebase (proyecto el-mercado10). El código se organiza en tres capas con dependencias dirigidas hacia el dominio.

Diagrama 1. Contexto del sistema

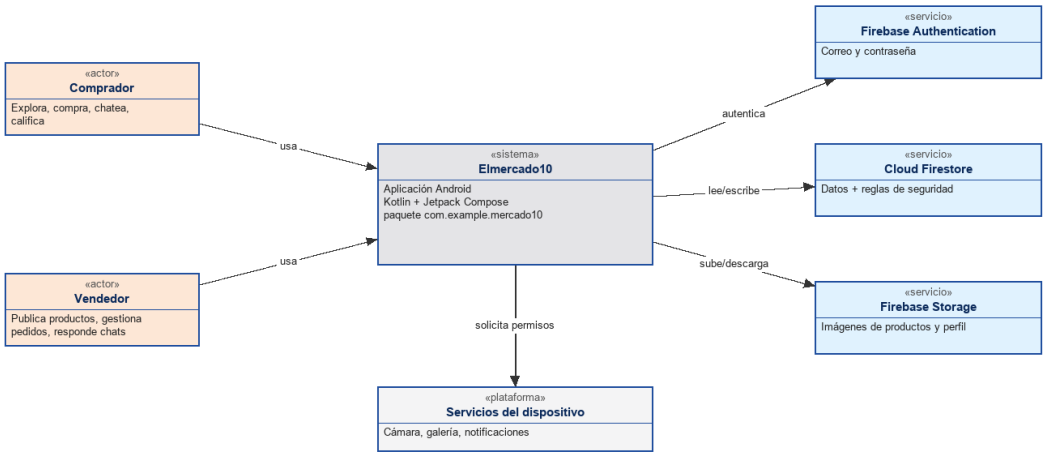


Figura 1. Contexto del sistema.

2. Capas

Diagrama 2. Arquitectura por capas (presentación, dominio, datos)

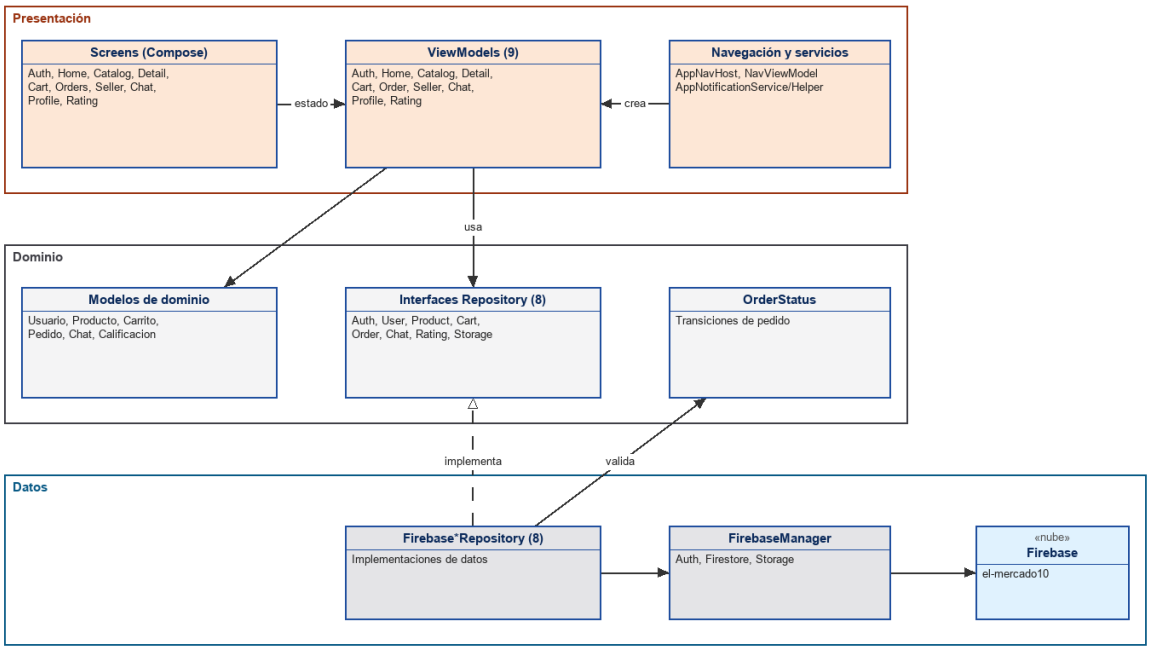


Figura 2. Arquitectura por capas.

Capa	Paquete	Contenido
Presentación	presentation (screens, components, viewmodel, navigation, notification)	Pantallas Compose, 9 ViewModels, AppNavHost, NavViewModel, servicio de notificaciones y AppContainer.
Dominio	domain (model, repository)	Seis grupos de modelos de datos, ocho interfaces de repositorio y OrderStatus.
Datos	data (firebase, repository)	FirebaseManager y ocho implementaciones Firebase*Repository.
Tema	ui.theme	Colores, tipografía y tema Material 3 en modo claro y oscuro.

3. Decisiones de diseño

ID	Decisión	Motivo	Consecuencia
AD-01	Repositorios definidos como interfaces de dominio.	Aislar la interfaz de Firebase y facilitar pruebas.	Cada ViewModel depende de una interfaz, no del SDK.
AD-02	Inyección manual con AppContainer.	Evitar una dependencia adicional en un proyecto de tamaño reducido.	Los repositorios se crean de forma perezosa y compartida.
AD-03	Compras y calificaciones en transacciones de Firestore.	Mantener coherentes inventario, pedido y promedio.	Todo se aplica o nada se aplica.
AD-04	Listeners en tiempo real expuestos como Flow.	Actualizar la interfaz sin recargar.	Cada listener debe cancelarse con el ciclo de vida.
AD-05	ViewModels ligados al back stack con navViewModel.	Cancelar listeners al salir de una pantalla y sobrevivir a rotaciones.	Se corrigió la fuga detectada con remember.
AD-06	Estados de pedido en un objeto de dominio y en las reglas.	Evitar que un cliente modificado salte el ciclo.	Las dos definiciones deben mantenerse alineadas.

4. Flujos principales

Diagrama 4. Flujo de compra y gestión de pedidos

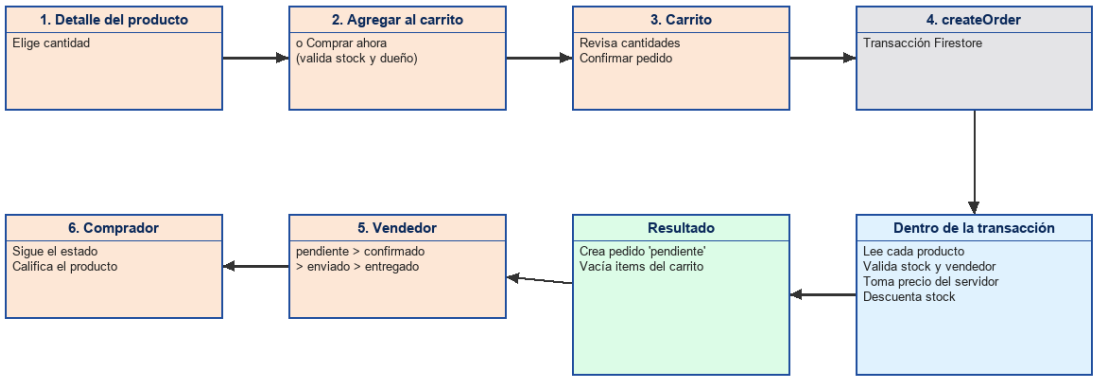


Figura 3. Flujo de compra.

Diagrama 5. Navegación (verde: solo rol vendedor)

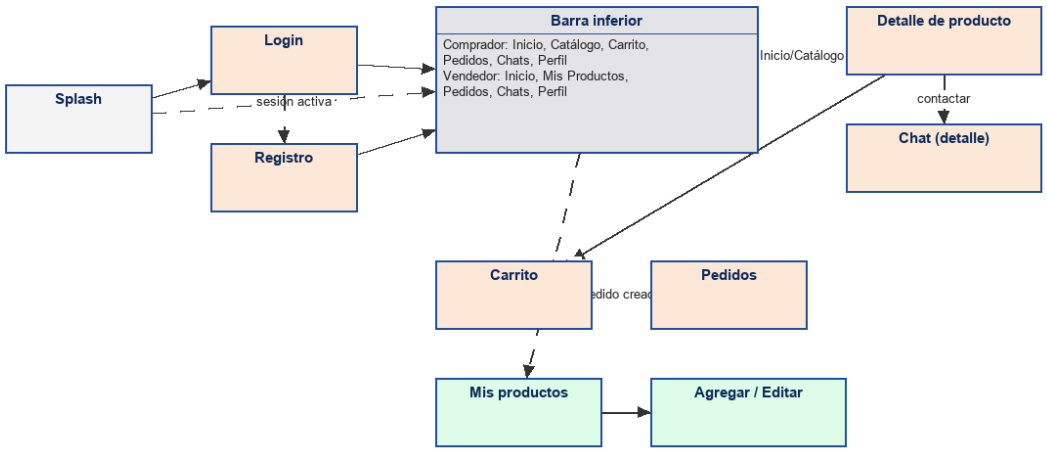


Figura 4. Navegación por rol.

5. Despliegue

Diagrama 6. Despliegue

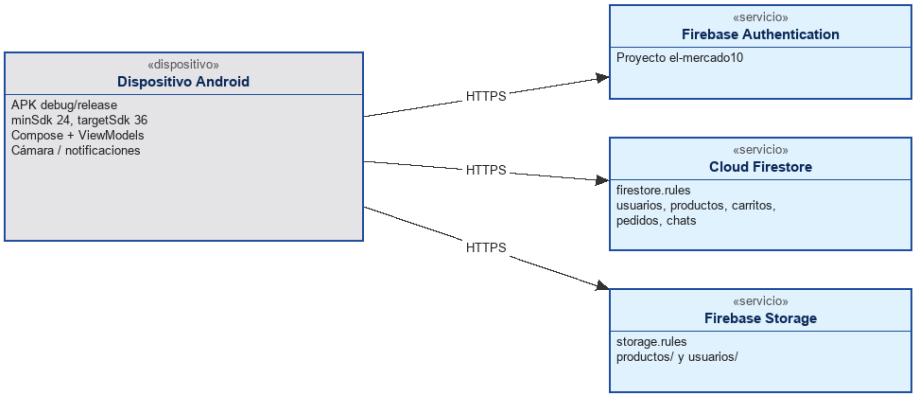


Figura 5. Despliegue.

6. Ciclo de vida de los ViewModels

Hasta esta revisión, los ViewModels se creaban con remember dentro de AppNavHost: nunca recibían onCleared y sus corrutinas seguían escuchando Firestore después de abandonar la pantalla. La función navViewModel los crea con viewModel() y una fábrica, de modo que su dueño es la entrada del back stack (o la actividad en el caso de la autenticación). Compila y no cambia la interfaz; su efecto en listeners abiertos queda pendiente de la prueba PP-021.